Part 1: Challenge (Node-RED)

Authors: Mostafa Moazenkakhki, Seyed Mohammad Tabatabayi

1. Periodic MQTT Message Generation

In this section, I implemented a Node-RED flow to periodically generate and publish MQTT messages every 5 seconds to a local Mosquitto MQTT broker.

Node-RED flow diagram:

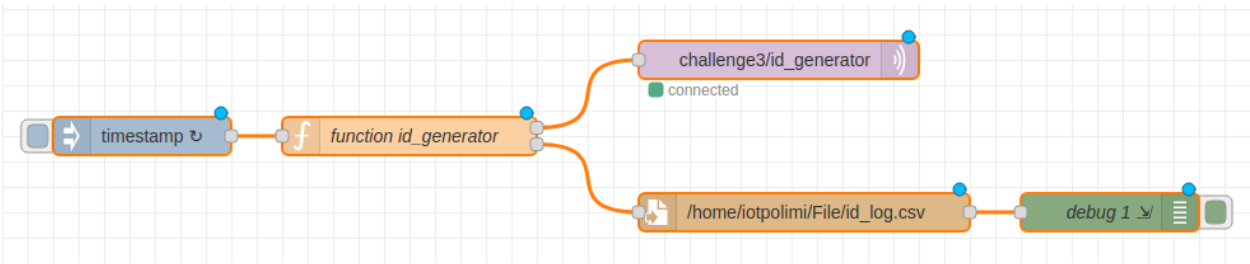
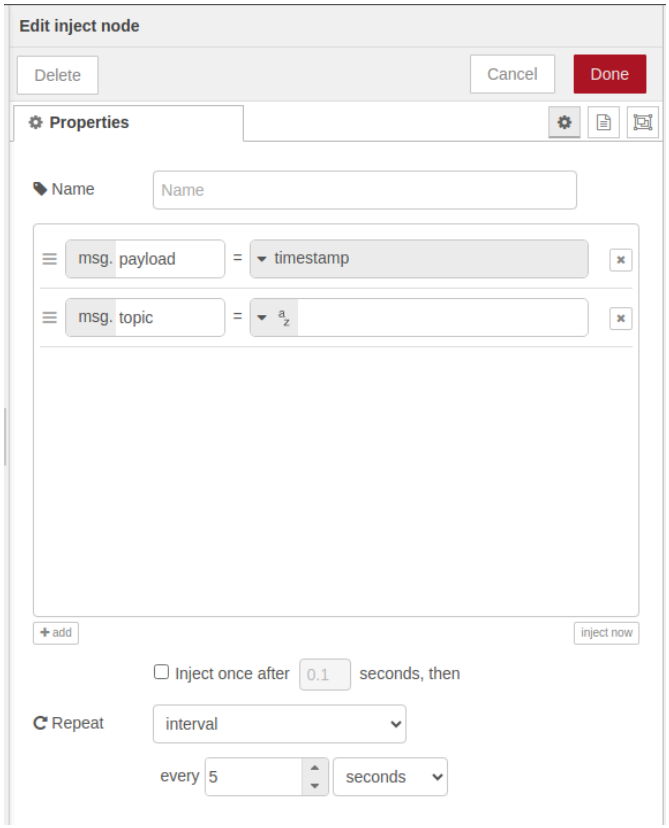


Figure 1: Node-RED flow diagram for periodic MQTT message publishing and logging.

Explanation of Nodes:

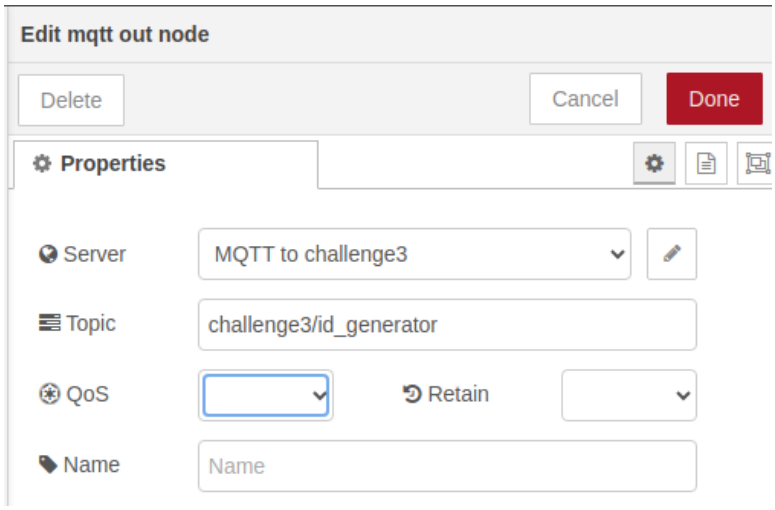
- **Inject node:** Set to trigger every 5 seconds, initiating the message generation.



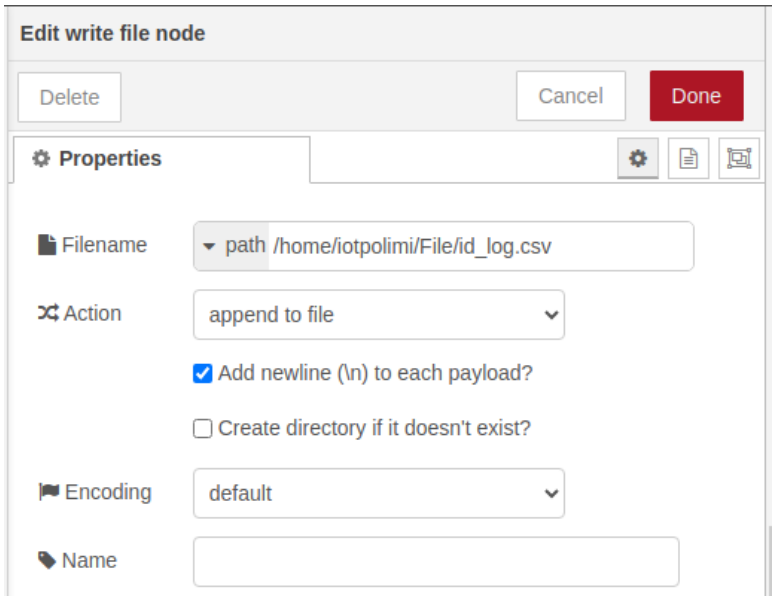
- **Function node:** Generates a random ID between 0 and 30000 and attaches a UNIX timestamp.

```
// row count
let count = flow.get("csv_count") || 1;
//generate payload
let id        = Math.floor(Math.random() * 30001);
let timestamp = Math.floor(Date.now() / 1000);

// prepare the two messages
let mqttMsg = { payload: { id, timestamp } };
let csvRow  = `${count},${id},${timestamp}`;       // no newline here
let csvMsg  = { payload: csvRow };

// bump counterflow.set("csv_count", count + 1);
//  return to outputs [ mqtt, csv ]
return [ mqttMsg, csvMsg ];
```

- **MQTT Out node:** Publishes the JSON message payload to the MQTT topic (`challenge3/id_generator`) on broker (localhost, port 1884).



- **File node (CSV Logging):** Logs messages into the CSV file `id_log.csv`.



Sample payload generated:

json

- `{"id": 18268, "timestamp":1745746907}`

CSV Logging Structure and samples that extracted comes below:

No.	ID	TIMESTAMP
1	1706	1745259582
2	17724	1745259587

2. MQTT Subscription and Message Processing

In this section, a Node-RED flow is designed to subscribe to MQTT messages published on the topic `challenge3/id_generator` from the local Mosquitto broker running on `localhost` at port `1884`.

Node-RED flow diagram:

Figure 2: Node-RED flow diagram illustrating MQTT subscription and message processing.

Explanation of Nodes:

- **MQTT In Node:** Configured to subscribe to `challenge3/id_generator` and listen for incoming MQTT messages.
- **Count & Gate:** it is check Idmsg for 80

```
// retrieve the count
```

```
let count = flow.get("idCount") || 0;

// increment and store
count += 1;
flow.set("idCount", count);

// gate based on 80
if (count <= 80) {
   return msg;
} else {
   // optionally warn once
   if (count === 81) node.warn("80-message limit reached, dropping further
msgs");
   return null;
}
```

- **Function Node (Compute N):** Upon receiving a message, it calculates the value `N` by computing the modulo of the received ID with `7711` (i.e., `N = ID % 7711`)

```
// Parse the incoming payload (from MQTT)
let data = typeof msg.payload === 'string' ? JSON.parse(msg.payload) :
msg.payload;

// Extract ID
let id = data.id;

// Compute N = id % 7711
let N = id % 7711;

// Store N in message for next node
msg.N = N;

return msg;
```

- **File Processing Node:** The computed `N` is used to process the provided `challenge3.csv` file. The flow locates and extracts the message that corresponds exactly to the frame number matching the computed `N`.

Calculation Example:

Received ID = 7781

Computed N = 7781 modulo 7711 = 70

CSV Processing:

- Messages are selected from `challenge3.csv` based on the computed frame number `N`.
- The processing step ensures correct identification and handling of relevant MQTT Publish or ACK messages for subsequent actions.

- **Function 1:** This node is responsible for matching **Id** with **the row of csv file** be provided to us.

```
let N = msg.N; // previously calculated frame number
let rows = msg.payload;

// Find row where frame number matches N
let result = rows.find(row => Number(row["No."]) === N);

msg.payload = result || `No row found for frame ${N}`;
return msg;
```

---

3. MQTT Publish Messages Handling

In this section, a Node-RED flow is implemented to process MQTT Publish messages identified from the `challenge3.csv` file. If a matching message is found, it is republished to the appropriate MQTT topic with a structured JSON payload.

Node-RED flow diagram:

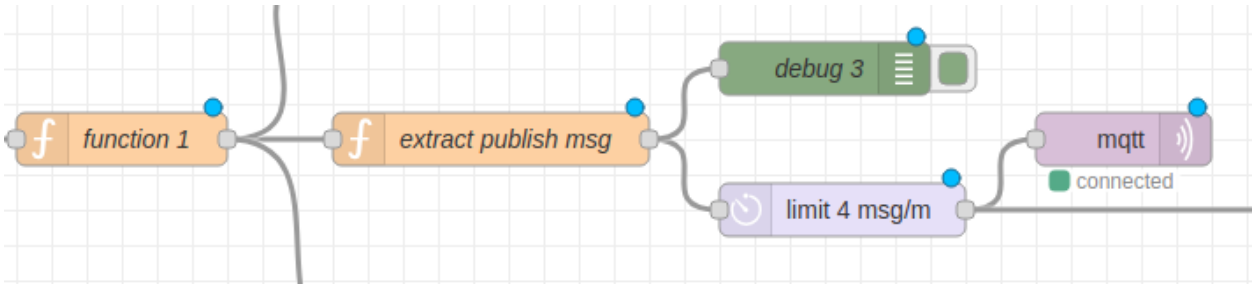


Figure 3: Node-RED flow diagram showing the handling and republishing of MQTT Publish messages.

Explanation of Nodes:

- **Function1:** After processing the CSV data, checks if the message type is an MQTT Publish.

```
let N = msg.N; // previously calculated frame number
let rows = msg.payload;

// Find row where frame number matches N
let result = rows.find(row => Number(row["No."]) === N);

msg.payload = result || `No row found for frame ${N}`;
return msg;
```

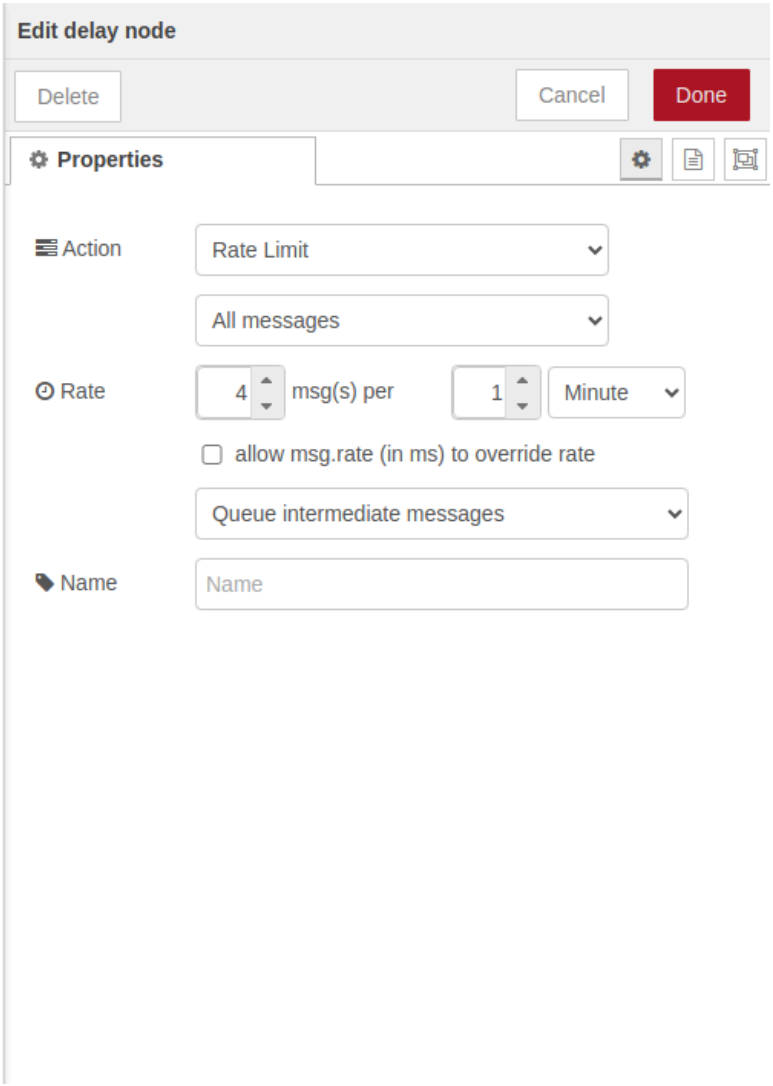
- **Function Node (Extract Publish msg):** Prepares a new JSON payload using the following fields:
    - `timestamp`: Current system time at the moment of publishing.
    - `id`: ID received from the subscription (original ID).

- `topic`: Topic extracted from the matched Publish message.
- `payload`: Payload content extracted from the Publish message.

```
// Build Publish Msgs
let row   = msg.payload||{};
let subId = msg.N;
let now   = Math.floor(Date.now()/1000);

// 1) detect Publish action
let info = row["Info"] || row["info"] || "";
if (!info.includes("Publish")) {
 node.warn(`Frame ${row["No."]||"?"}: no Publish action`);
 return null;
}

// 2) figure out which column name holds the raw payload
// adjust these to whatever your inspection shows
let rawData = (row["Payload"] || row["payload"] || "").trim();

// 3) extract topic(s) from the Info field
let topics = [];
for (let m of info.matchAll(/\[([^\]]+)\]/g)) {
 topics.push(m[1]);
}

// 4) parse the payload JSON (or fallback to empty object)
let payloads;
try {
 payloads = JSON.parse(rawData);
} catch(e) {
 node.warn(`Invalid JSON in payload for frame ${msg.N}: ${e.message}`);
 payloads = {};
}
if (!Array.isArray(payloads)) payloads = [payloads];
// 5) build one msg per publish entry
let out = [];
let count = Math.max(topics.length, payloads.length);
for (let i = 0; i < count; i++) {
 let t = topics[i]     || topics[0];
 let p = payloads[i]   || {};
 out.push({
   topic: t,
   payload: JSON.stringify({
```

```
      timestamp: now,
      id:        subId,
      topic:     t,
      payload:   p
    })
  });
}

return out;
```

- **Rate Limiter Node:** Ensures that a maximum of **four Publish messages per minute** are sent, complying with the delivery requirements.



Structure of JSON Payload Example:

```
{
  "timestamp": "1712561821",
  "id": "7747",
  "topic": "hospital/facility1",
  "payload": {
    "long": 80,
    "range": [0,59],
    "lat": 86,
    "type": "temperature",
    "unit": "C",
    "description": "Room Temperature"
  }
}
```

Perfect, let's continue! 🚀
Here's the full, clean, and professional writing for **Part 4: Temperature Data Visualization and Logging**, following exactly the same structure:

---

4. Temperature Data Visualization and Logging

In this section, the Node-RED flow was expanded to process temperature data from Publish messages and visualize the filtered data on a live chart. Only temperature values measured in Fahrenheit were considered for visualization and CSV logging.

Node-RED flow diagram:

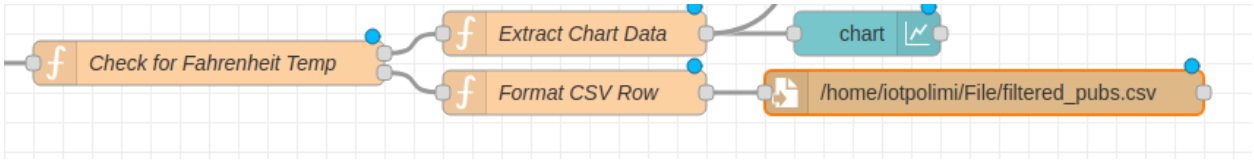


Figure 4: Node-RED flow diagram for filtering, processing, and visualizing temperature messages.

Explanation of Nodes:

- **Decision Node (Filter Fahrenheit Messages):** After a Publish message is detected, the payload is checked for:
    - `type` = "temperature"
    - `unit` = "F"

```
let payload = msg.payload;
let filteredMessages = [];

try {
   if (typeof payload === "string") payload = JSON.parse(payload);
   if (typeof payload.payload === "string") payload.payload =
JSON.parse(payload.payload);

   const items = Array.isArray(payload.payload) ? payload.payload :
[payload.payload];

   items.forEach((item) => {
       if (item.unit === "F" && item.type === "temperature") {
           const mean = (item.range[0] + item.range[1]) / 2;
           filteredMessages.push({
               long: item.long,
               lat: item.lat,
               mean: mean,
               type: item.type,
               unit: item.unit,
               description: item.description
           });
       }
   });
} catch (e) {
   node.warn("Error parsing message: " + e);
}

if (filteredMessages.length > 0) {
   msg.filtered = filteredMessages;
```

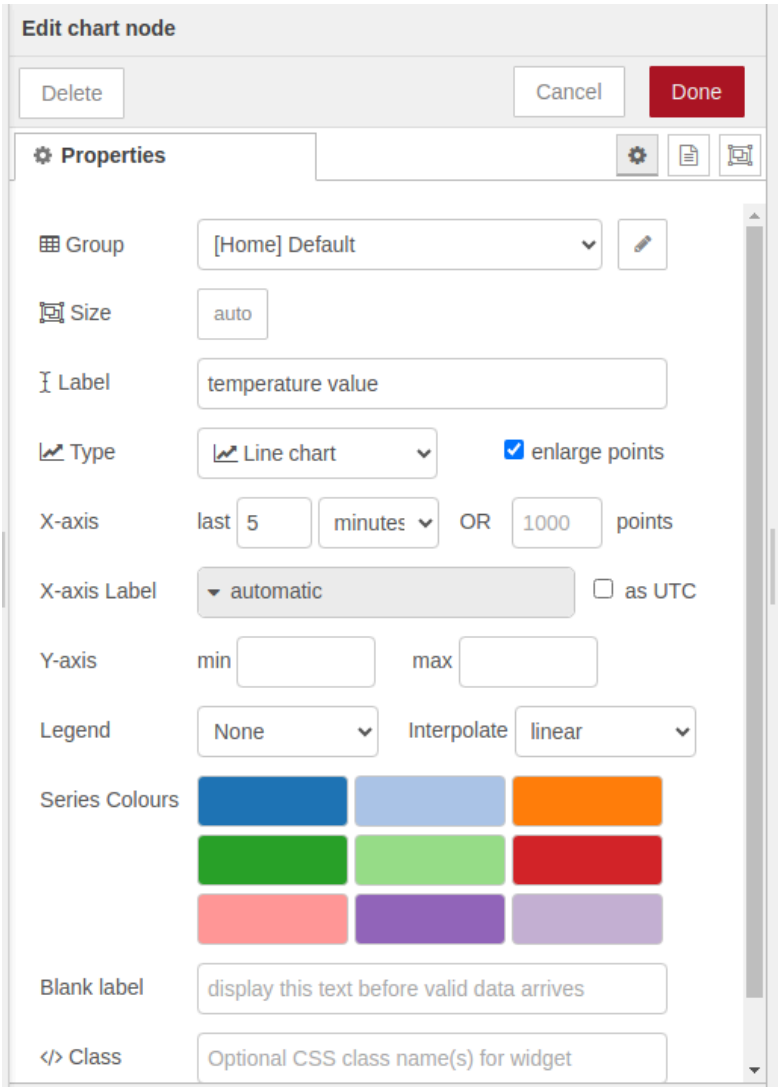
```
   return [msg, msg]; // to both chart and CSV
}
return [null, null];
```

- **Function Node (Calculate Mean Value):** Computes the mean value of the temperature from the `range` array [min, max]:

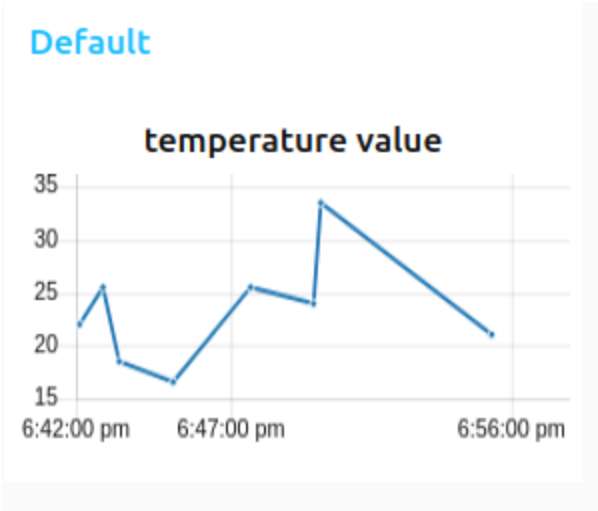
$$\text{mean } value = \frac{\min + \max}{2}$$

```
msg.filtered.forEach(f => {
   node.send({ payload: f.mean });
});
return null;
```

- **Chart Node:** Plots the temperature values (mean values) on a real-time Node-RED chart.


Edit chart node
Delete
Cancel
Done
Properties
Group
[Home] Default
Size
auto
Label
temperature value
Type
Line chart
enlarge points
X-axis
last
5
minutes
OR
1000
points
X-axis Label
automatic
as UTC
Y-axis
min
max
Legend
None
Interpolate
linear
Series Colours
Blank label
display this text before valid data arrives
Class
Optional CSS class name(s) for widget

the result :



CSV Logging Node: Saves the filtered temperature messages into a `filtered_pubs.csv` file with the following fields:

```
let counter = flow.get("csv_counter") || 1;
let rows = [];

msg.filtered.forEach((f) => {

rows.push(`${counter},${f.long},${f.lat},${f.mean},${f.type},${f.unit},${f.desc
ription}`);
   counter++;
});

flow.set("csv_counter", counter);

msg.payload = rows.join("\n");
return msg;
```

No., LONG, LAT, MEAN_VALUE, TYPE, UNIT, DESCRIPTION

Example Processed Data for CSV:

No.	LONG	LAT	MEAN_VALUE	TYPE	UNIT	DESCRIPTION
1	92	80	22.5	temperature	F	Room Temperature

Important Notes:

- Only messages that pass the filtering criteria are plotted and logged.
- If a Publish packet contains multiple payloads (e.g., multiple rooms), each qualifying payload must be handled separately.🚀)

Awesome, let's keep going! 🚀
Here's the complete, clean, and professional **Part 5: MQTT ACK Message Handling**, in the same style:

---

5. MQTT ACK Message Handling

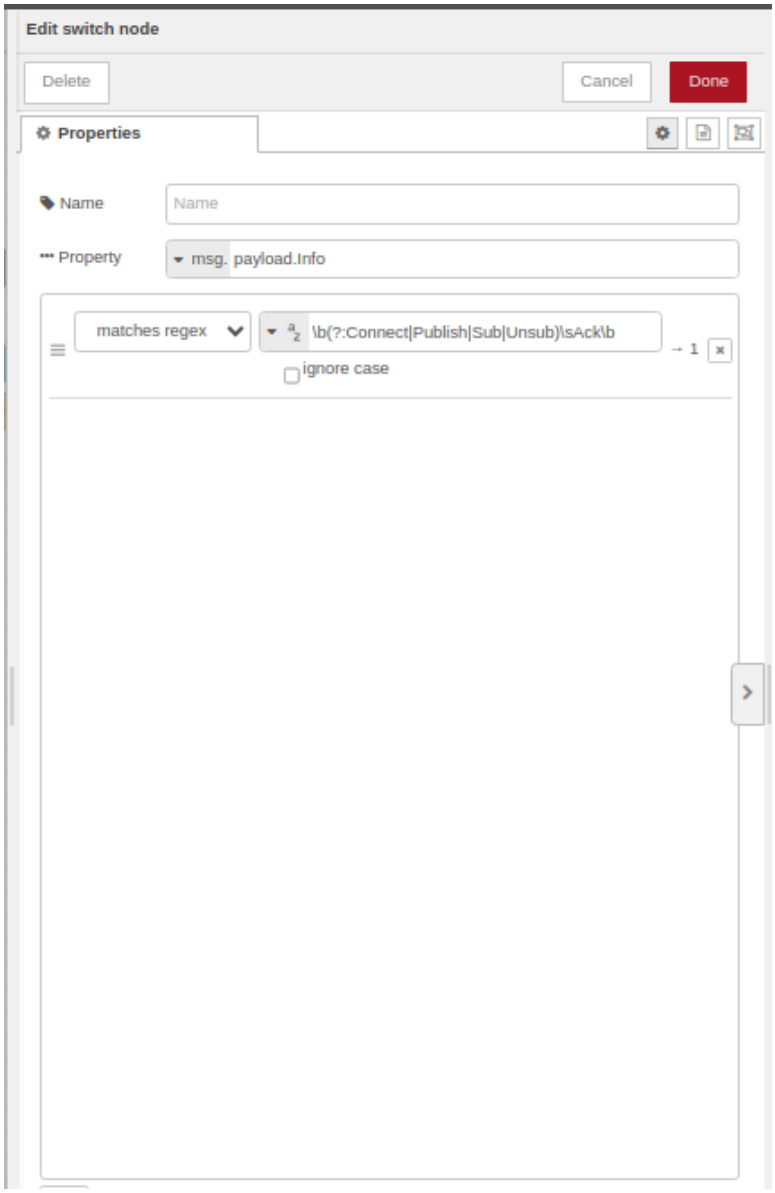
In this section, the Node-RED flow handles MQTT ACK messages identified in the `challenge3.csv` file. Upon detection of an ACK message, the flow updates a global counter and logs the acknowledgment information into a dedicated CSV file.

Node-RED flow diagram:

Figure 5: Node-RED flow diagram showing the handling and logging of MQTT ACK messages.

Explanation of Nodes:

- **Decision Node (Detect ACK Messages):** After processing the incoming data, checks if the message type corresponds to an ACK message (Publish Ack, Connect Ack, Sub/Unsub Ack).



- **Counter Node:** Increments a global counter every time an ACK message is detected.

```
// pull the counter from nem store
let ackCounter = global.get("ack_counter","nem") || 1;

// timestamp
```

```
let timestamp = Date.now();

// extract the subId from the parsed CSV row:
let subId = msg.payload["No."] || msg.payload.No || "UNKNOWN";

// msgType as before
let msgType = msg.payload.Info;

// build CSV row
let csvRow = `${ackCounter},${timestamp},${subId},${msgType}`;

// increment and save
global.set("ack_counter", ackCounter+1, "nem");

// set outputs
msg.payload  = csvRow;
msg.ackValue = ackCounter;
return msg;
```

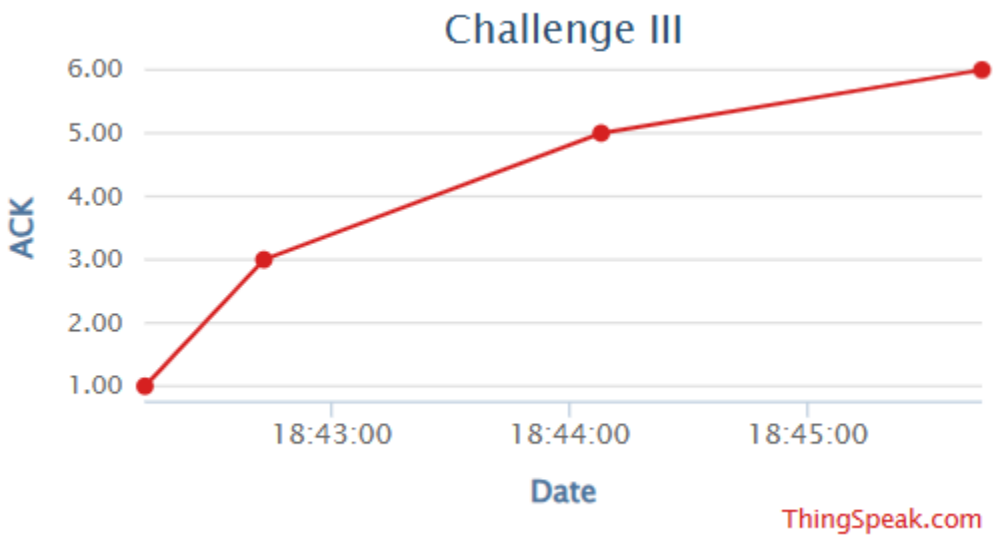
CSV Logging Node: Logs the ACK message details into the `ack_log.csv` file with the following fields:

No., TIMESTAMP, SUB_ID, MSG_TYPE

- **HTTP Request Node:** Sends the current value of the global ACK counter to a public ThingSpeak channel using an HTTP POST API call.

```
const apiKey    = "H2N0EELN1ZMUTKHE";
const channelId = "2928185";

// ackValue was set in Handle ACK Message node
const ackCount = msg.ackValue;

// Build the GET request
msg.method = "GET";
msg.url    = `https://api.thingspeak.com/update`
           + `?api_key=${apiKey}`
           + `&field1=${ackCount}`;

return msg;
```

the result:



Example Log Entry for `ack_log.csv`:

No.	TIMESTAMP	SUB_ID	MSG_TYPE
1	1.75E+12	534	Publish Ack (id=7)
2	1.75E+12	6190	Publish Ack (id=1)

Example HTTP POST to ThingSpeak:

POST https://api.thingspeak.com/update

?api_key=YOUR_WRITE_API_KEY &field1=ACK_COUNTER_VALUE

- Make sure that your ThingSpeak channel is set to **public**.
- Use the **Field 1** to record the ACK counter value.

Important Notes:

- Only ACK messages increment the counter and trigger updates to ThingSpeak.
- Ensure accurate time-stamping when logging the ACK messages.
- Manage API request frequency carefully to avoid exceeding ThingSpeak limits.

---

6. Flow Control and Message Limits

This section describes how the Node-RED flow was designed to control the overall message processing, ensuring the system stops after handling exactly 80 valid ID messages, as per the challenge requirements.

Node-RED flow diagram:

Figure 6: Node-RED flow diagram illustrating flow control and limiting the number of processed messages.

Explanation of Nodes:

- **Counter and gate node (Valid Messages Counter):** Increments a counter only when a valid message (Publish or ACK) is processed

```
let count = flow.get("idCount") || 0;

// increment and store
count += 1;
flow.set("idCount", count);

// gate based on 80
if (count <= 80) {
   return msg;
} else {
   // optionally warn once
   if (count === 81) node.warn("80-message limit reached, dropping further
msgs");
   return null;
}
```